

Learning PyTorch: The 20% That Matters

1 The Big Picture

Every deep learning project in PyTorch follows the same loop: get data into **tensors**, push them through a **model**, compute a **loss**, call **backward()** to get gradients, and have an **optimizer** update the weights. Master this loop and you understand 80% of PyTorch.

2 Tensors: The Foundation

A tensor is an n-dimensional array (like NumPy) that can live on a GPU and track gradients.

```
1 import torch
2
3 # Creating tensors
4 x = torch.tensor([[1., 2.], [3., 4.]]) # from data
5 z = torch.zeros(3, 4)                  # shape (3,4)
6 o = torch.ones(2, 3)
7 r = torch.randn(2, 3)                  # normal distribution
8
9 # Key attributes
10 x.shape      # torch.Size([2, 2])  <-- ALWAYS check this
11 x.dtype      # torch.float32
12 x.device     # cpu or cuda:0
13
14 # Reshaping (critical for matching layer shapes)
15 r.view(3, 2) # reshape, must keep same number of elements
16 r.reshape(6)
17 r.unsqueeze(0) # add a dim -> shape (1, 2, 3)
18 r.squeeze()   # remove size-1 dims
19 r.permute(1, 0) # swap axes
```

3 Shapes Are Everything

Most PyTorch bugs are shape mismatches. Convention for a batch is (N, ...) where N is batch size. For example a batch of images is (N, C, H, W) = (batch, channels, height, width). A batch of text token embeddings is (N, L, D) = (batch, sequence length, embedding dim). **Print .shape constantly while debugging.**

4 GPU Usage

```

1 device = "cuda" if torch.cuda.is_available() else "cpu"
2 model = model.to(device)
3 x = x.to(device)           # data and model MUST be on the same device

```

5 Autograd (Automatic Differentiation)

```

1 x = torch.tensor(2.0, requires_grad=True)
2 y = x**2 + 3*x
3 y.backward()           # computes dy/dx
4 print(x.grad)          # tensor(7.) since dy/dx = 2x+3 = 7

```

You rarely call this manually — the training loop does it for you. The point: tensors with `requires_grad=True` build a graph, and `backward()` fills in `.grad`.

6 Building a Model

Subclass `nn.Module`. Define layers in `__init__`, define the forward pass in `forward`.

```

1 import torch.nn as nn
2
3 class Net(nn.Module):
4     def __init__(self):
5         super().__init__()
6         self.fc1 = nn.Linear(784, 128) # in_features, out_features
7         self.fc2 = nn.Linear(128, 10)
8         self.relu = nn.ReLU()
9
10    def forward(self, x):
11        x = self.relu(self.fc1(x))
12        x = self.fc2(x)
13        return x
14
15 model = Net()

```

7 Common Layers

- `nn.Linear(in, out)` — fully connected layer.
- `nn.Conv2d(in_ch, out_ch, kernel_size)` — convolution for images.
- `nn.ReLU()`, `nn.Sigmoid()`, `nn.Tanh()` — activations.
- `nn.Dropout(p)` — regularization.
- `nn.BatchNorm2d(n)` — normalization.
- `nn.Embedding(num, dim)` — lookup table for tokens.

8 Loss Functions and Optimizers

```
1 # Classification
2 criterion = nn.CrossEntropyLoss()
3 # Regression
4 criterion = nn.MSELoss()
5
6 # Optimizer (Adam is the safe default)
7 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

9 THE Training Loop (Memorize This)

```
1 for epoch in range(num_epochs):
2     for xb, yb in dataloader:
3         xb, yb = xb.to(device), yb.to(device)
4
5         optimizer.zero_grad()      # 1. clear old gradients
6         preds = model(xb)          # 2. forward pass
7         loss = criterion(preds, yb) # 3. compute loss
8         loss.backward()            # 4. backprop (compute gradients)
9         optimizer.step()           # 5. update weights
10
11     print(f"epoch {epoch}: loss={loss.item():.4f}")
```

These 5 steps are the heart of PyTorch. The order matters: `zero_grad` → `forward` → `loss` → `backward` → `step`.

10 Data Loading

```
1 from torch.utils.data import DataLoader, TensorDataset
2
3 dataset = TensorDataset(X_tensor, y_tensor)
4 dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
```

11 Evaluation Mode

```
1 model.eval()      # turns off dropout/batchnorm updates
2 with torch.no_grad(): # disables gradient tracking (faster)
3     preds = model(test_x)
4 model.train()      # switch back for training
```

12 Saving and Loading

```
1 # Save just the weights (recommended)
2 torch.save(model.state_dict(), "model.pth")
3
```

```
4 # Load
5 model = Net()
6 model.load_state_dict(torch.load("model.pth"))
7 model.eval()
```

13 Practice Task

Build a small `nn.Module` for classifying random data, write the full training loop from memory, then run an evaluation pass inside `torch.no_grad()`. Print `.shape` at every step until it feels natural.